ФГАОУ ВО «Омский государственный технический университет»
Кафедра «Автоматизированные системы обработки информации и управления»

ОТЧЁТ

о лабораторной работе № 3 по дисциплине
«Кроссплатформенная разработка»
студента Рябченко Ивана Сергеевича группы ПИН-252т

Пояснительная записка

Шифр работы ЛР–02068999–ПИН-20-25 ПЗ
Направление 09.03.04 Программная инженерия

Преподаватель А. А. Сафронов

Студент И. С. Рябченко

Омск 2026

ЗАДАНИЕ

Тема работы: сеть в приложении «Публикации (OpenAIRE)» на Flutter – замена моков второй работы данными из OpenAIRE Graph API.

Что нужно сделать:

– API-клиент на dio, датасорс, DTO на json_serializable и маппер в доменную модель;

– цепочку API-клиент → датасорс → репозиторий → Cubit или Bloc → экран;

– индикатор загрузки на списке и на карточке, у карточки свой запрос;

– сообщение об ошибке и кнопку «Повторить», если сети нет или сервер не отвечает;

– поиск на сервере с подписью «ничего не найдено»;

– подгрузку следующей страницы при прокрутке;

– параллельную загрузку похожих публикаций.

Приложение должно работать в браузере и ещё на одном таргете, вторым таргетом выбран macOS.

РЕФЕРАТ

Отчёт 15 с., 9 рис., 1 табл., 5 источн.

FLUTTER, DART, DIO, JSON_SERIALIZABLE, DTO, REST API, FLUTTER_BLOC, ОБРАБОТКА ОШИБОК, ПАГИНАЦИЯ, OPENAIRE GRAPH

Объект разработки – приложение-каталог научных публикаций, которое получает данные из OpenAIRE Graph API.

Цель работы – заменить моки второй работы сетью: получать список, результаты поиска и карточку публикации с сервера и показывать загрузку и ошибки.

В результате приложение загружает публикации из OpenAIRE Graph API, ищет по названию на сервере, подгружает страницы при прокрутке и показывает карточку с похожими публикациями. При ошибке сети выводится сообщение и кнопка «Повторить». Приложение работает в браузере и на macOS.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 5
1 Источник данных и запросы 6
2 Цепочка слоёв 6
3 Обработка ошибок 7
4 Список: загрузка, поиск и страницы 8
5 Карточка и похожие публикации 8
6 Проверка работы 9
ЗАКЛЮЧЕНИЕ 14
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 15

ВВЕДЕНИЕ

Курс построен вокруг одного приложения, которое растёт от работы к работе. Во второй работе сделан каркас каталога публикаций на Flutter, данные в нём лежали в коде. Вторая веха курса посвящена данным: в этой работе каталог получает записи из сети.

Цель работы – заменить моки сетью на Flutter [1] так, чтобы экраны и состояния второй работы сохранили своё устройство, а основные изменения пришлись на папку data. Данные берутся из OpenAIRE Graph API [4] через HTTP-клиент dio [2].

Задачи работы:

– подключить dio и описать ответы сервера через DTO на json_serializable;

– сделать API-клиент, датасорс и маппер DTO в модель;

– перевести репозиторий с моков на датасорс;

– перевести ошибки сети в свои типы и показать их на экране с кнопкой «Повторить»;

– сделать поиск на сервере и подгрузку страниц в Cubit списка;

– загружать карточку и похожие публикации в Bloc карточки;

– проверить работу приложения в браузере и на macOS.

1 Источник данных и запросы

OpenAIRE Graph – открытый граф научных результатов: статей, датасетов и программ, собранных из репозиториев и журналов [4]. У графа есть REST API версии 1 по адресу https://api.openaire.eu/graph/v1. Ключ для чтения не нужен, сервер отдаёт заголовок CORS, поэтому запросы работают и из браузера. Лимит – около 7200 запросов в час.

Все записи лежат в одной коллекции researchProducts. Без фильтра первые страницы заполнены записями типа other, поэтому каждый запрос списка уходит с параметром type=publication,dataset,software. Приложение делает четыре вида запросов, они приведены в таблице 1.

Таблица 1 – Запросы к OpenAIRE Graph API

Действие	Адрес	Параметры
Список	GET /researchProducts	type, page, pageSize=20
Поиск	GET /researchProducts	type, page=1, pageSize=20, mainTitle
Карточка	GET /researchProducts/{id}	–
Похожие	GET /researchProducts	type, page=1, pageSize=5, subjects; до трёх запросов параллельно

Модель PublicationModel со второй работы почти не изменилась: названия полей и раньше были взяты из API. Поля publicationDate, description и publisher стали необязательными, потому что у части записей их нет. Файл с моками удалён.

2 Цепочка слоёв

Данные проходят цепочку API-клиент → датасорс → репозиторий → Cubit или Bloc → экран. Новые классы лежат в папке data фичи publication:

– api – PublicationApi, один метод на один адрес сервера, ответ разбирается в DTO;

– data_sources – интерфейс IPublicationDataSource и NetworkPublicationDataSource, который вызывает API-клиент и переводит ошибки dio в LoadException;

– dto и mappers – PublicationDto с вложенными AuthorDto и SubjectDto и расширение toModel, которое переводит DTO в модель.

DTO описаны через json_serializable [3], код разбора генерирует build_runner, файлы .g.dart лежат в git. В build.yaml включён режим checked: если

поле пришло не того типа, разбор бросает CheckedFromJsonException. Маппер убирает HTML-разметку из названия и описания, лишние запятые в именах авторов и пустые значения.

Dio создаётся один раз в DioClient: базовый адрес, таймаут подключения 10 с и ответа 20 с, в отладочной сборке LogInterceptor пишет запросы в консоль. Вся цепочка собирается в app.dart, там же, где во второй работе стоял RepositoryProvider:

```
RepositoryProvider<IPublicationRepository>(
  create: (_) => PublicationRepository(
    NetworkPublicationDataSource(
      PublicationApi(DioClient.create()),
    ),
  ),
),
```

Cubit и Bloc получают репозиторий через context.read по интерфейсу, как и во второй работе, поэтому про dio и DTO они не знают.

3 Обработка ошибок

Все ошибки загрузки описаны sealed-классом LoadException в папке common/network. У него пять вариантов: NoConnectionException – нет сети или сервер недоступен, TimeoutLoadException – сервер не ответил вовремя, NotFoundException – ответ 404, ServerException – другой код ошибки, BadResponseException – ответ не удалось разобрать.

Перевод делает датасорс в методе _guard: запрос выполняется внутри try, тип DioException разбирается через switch, а CheckedFromJsonException и FormatException превращаются в BadResponseException. Выше датасорса про dio никто не знает: репозиторий пропускает LoadException наверх, а Cubit и Bloc ловят только его.

Экран показывает ошибку виджетом LoadErrorView. Он выбирает по типу ошибки фразу из ARB, а не текст исключения, и рисует кнопку «Повторить» (рисунок 7). Для NotFoundException кнопки нет: записи, которой нет на сервере, не появится. Подписи ошибок добавлены в оба ARB-файла, теперь в каждом 20 ключей.

4 Список: загрузка, поиск и страницы

Состояние списка – sealed-класс с тремя вариантами: PublicationListLoading, PublicationListLoaded и PublicationListFailure, запрос поиска хранится в каждом из них. При запуске Cubit выдаёт состояние загрузки, экран показывает индикатор, затем список из 20 записей (рисунок 1). При ошибке выводится LoadErrorView, кнопка «Повторить» вызывает метод retry, который повторяет последний запрос (рисунки 7 и 8).

Поиск идёт на сервере по параметру mainTitle. Чтобы не отправлять запрос на каждую букву, метод search ждёт 400 мс после последнего изменения текста. Ответ на старый запрос может прийти позже нового, поэтому после каждого await Cubit проверяет, что он не закрыт и что запрос в состоянии тот же, иначе ответ отбрасывается. Если ничего не нашлось, выводится подпись из второй работы (рисунок 3).

Когда до конца списка остаётся меньше 240 пикселей, экран вызывает метод loadMore. Номер следующей страницы считается по числу уже загруженных записей, поэтому повторов нет. Пока страница грузится, внизу списка крутится индикатор (рисунок 2), повторные вызовы в это время отсекаются. Если страница не загрузилась, показанные записи остаются, а внизу появляется сообщение с кнопкой «Повторить». У результатов поиска страниц нет, сервер отдаёт первые 20 совпадений.

5 Карточка и похожие публикации

Карточка работает на Bloc [5], как во второй работе. Экран карточки создаёт Bloc и сразу отправляет событие PublicationDetailOpened с идентификатором из адреса, поэтому карточка грузится своим запросом и со своим индикатором (рисунок 4). Событие PublicationDetailRetried отправляет кнопка «Повторить», оба события обрабатывает один метод.

Состояния карточки: PublicationDetailLoading, PublicationDetailLoaded, PublicationDetailNotFound и PublicationDetailFailure. Если сервер ответил 404, Bloc выдаёт NotFound, и экран пишет, что публикации в каталоге нет (рисунок 6). Если карточку закрыли раньше, чем пришёл ответ, Bloc уже закрыт и новое состояние не выдаётся, поэтому ошибок в консоли нет.

Похожие публикации подбирает репозиторий в методе getDetails. Сначала загружается сама запись, затем по трём её первым тематикам уходят три запроса

по пять записей. Запросы не зависят друг от друга и идут параллельно через Future.wait. Результаты объединяются без повторов, сама запись из них исключается (рисунок 5). Если не загрузились похожие, карточка показывает ошибку целиком, потому что они грузятся вместе с публикацией в getDetails.

6 Проверка работы

Приложение запускается в браузере Chrome и на macOS. Для macOS в файлы entitlements добавлено право com.apple.security.network.client, без него песочница не пускает приложение в сеть. Команды:

```
flutter run -d chrome
flutter run -d macos
dart run build_runner build --delete-conflicting-outputs
```

Анализатор flutter analyze замечаний не выдаёт, скрипт check_arb расхождений в ARB не находит. Все пункты проверки из задания выполнены: четыре обязательных и три желательных.

Ниже приведены экраны приложения в браузере. Для рисунка 7 сеть отключалась подменой адреса сервера. На рисунке 9 вкладка Network: запрос карточки и три параллельных запроса похожих публикаций к api.openaire.eu.

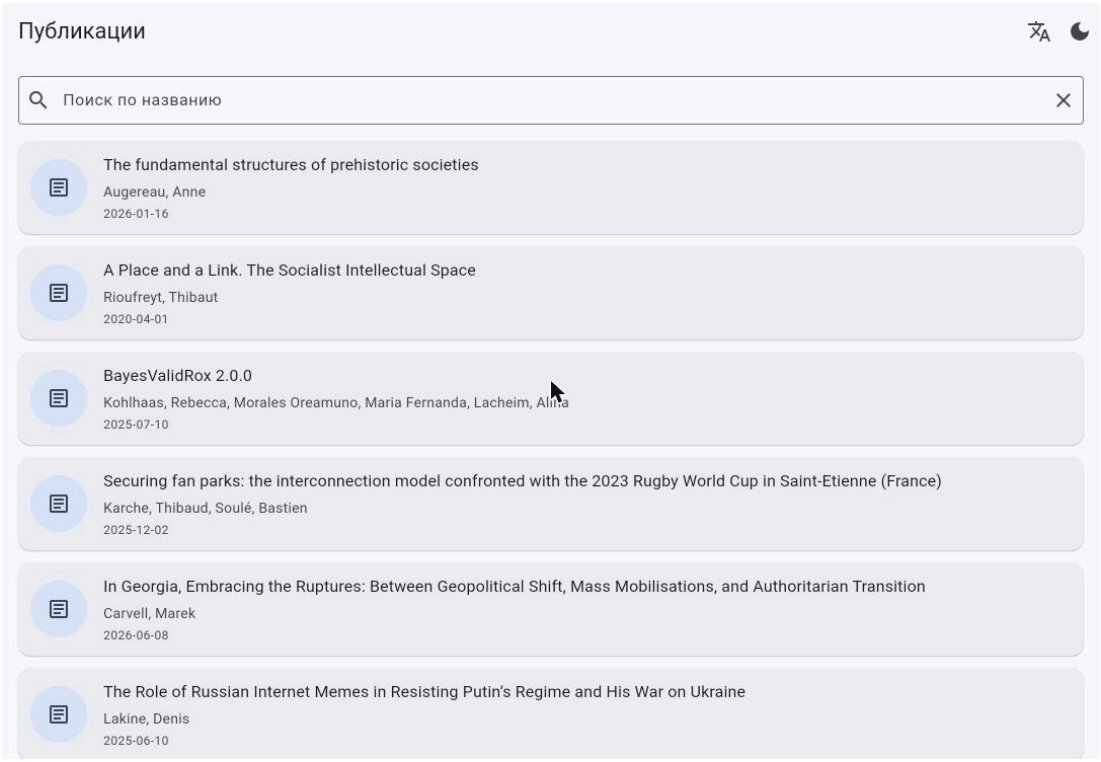


Рисунок 1 – Список публикаций из сети

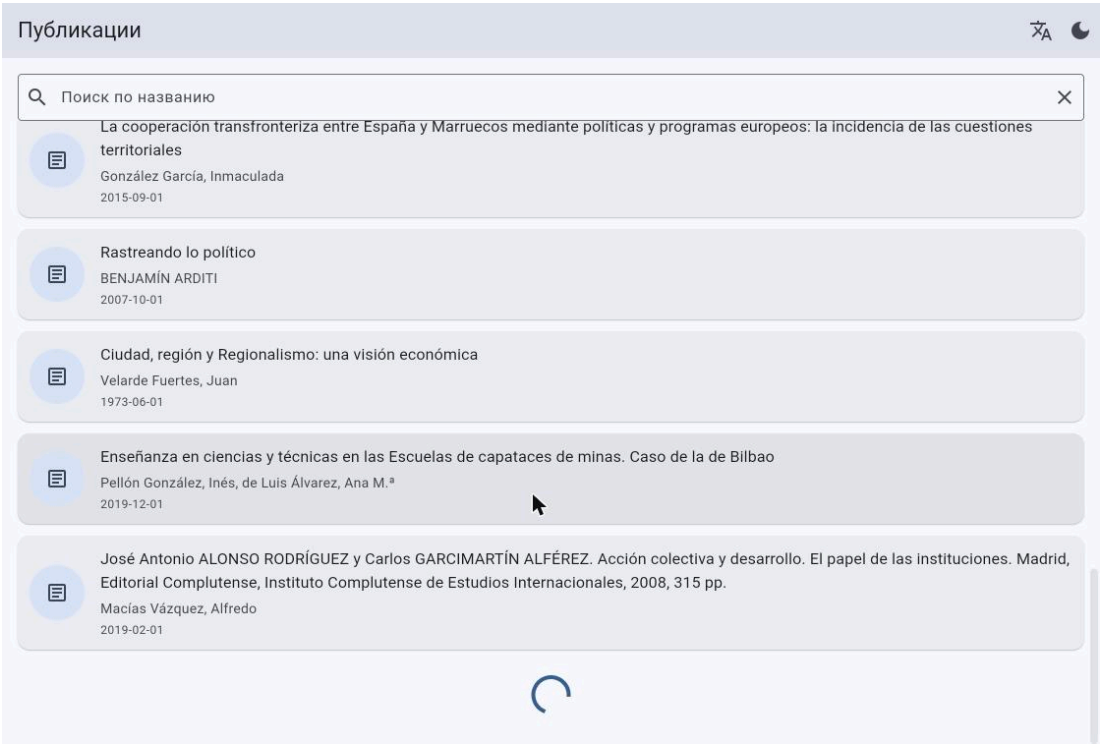


Рисунок 2 – Подгрузка следующей страницы

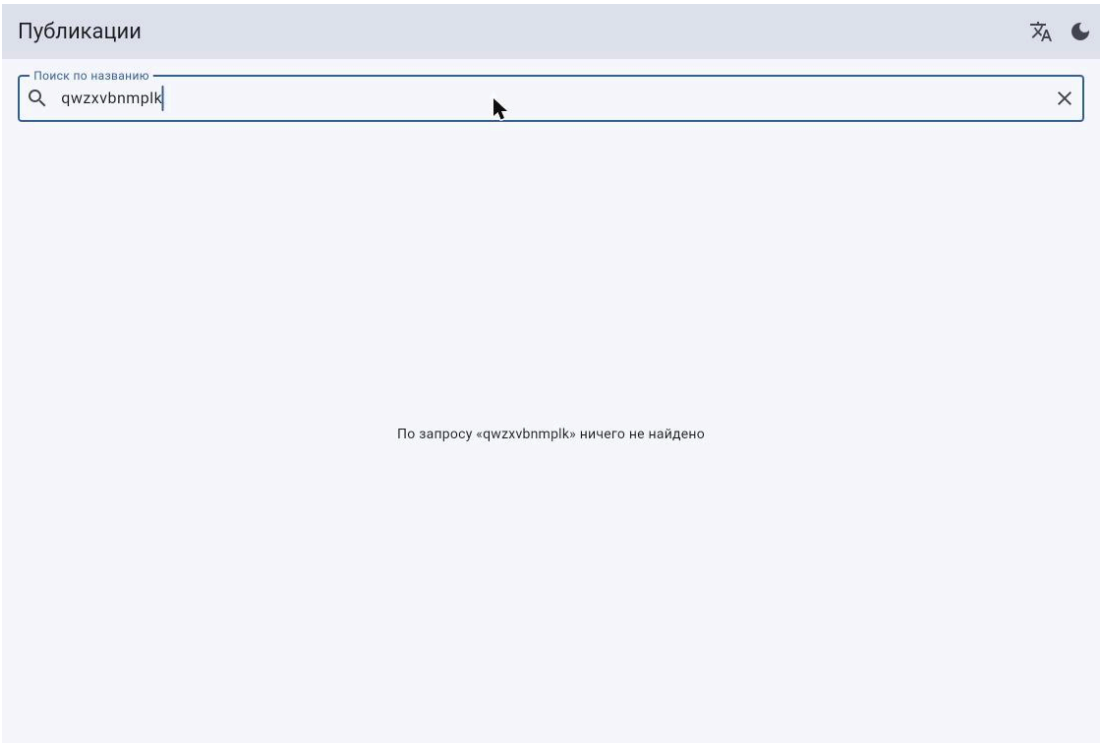


Рисунок 3 – Поиск без результатов

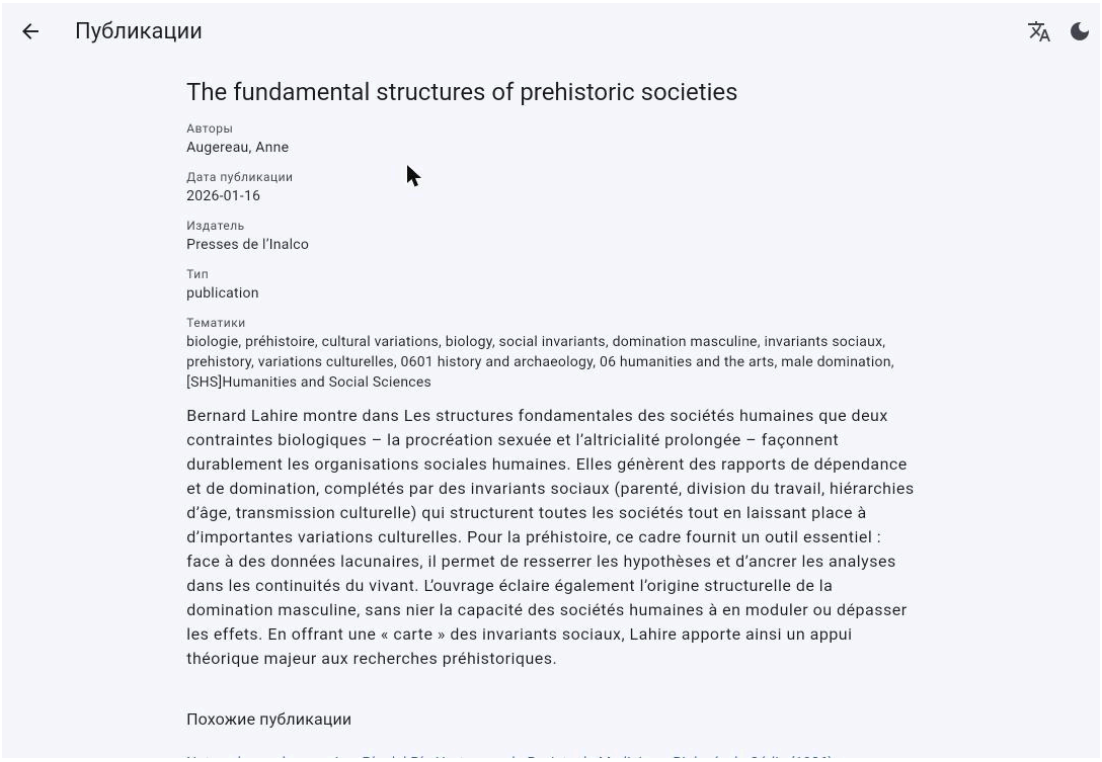


Рисунок 4 – Карточка публикации

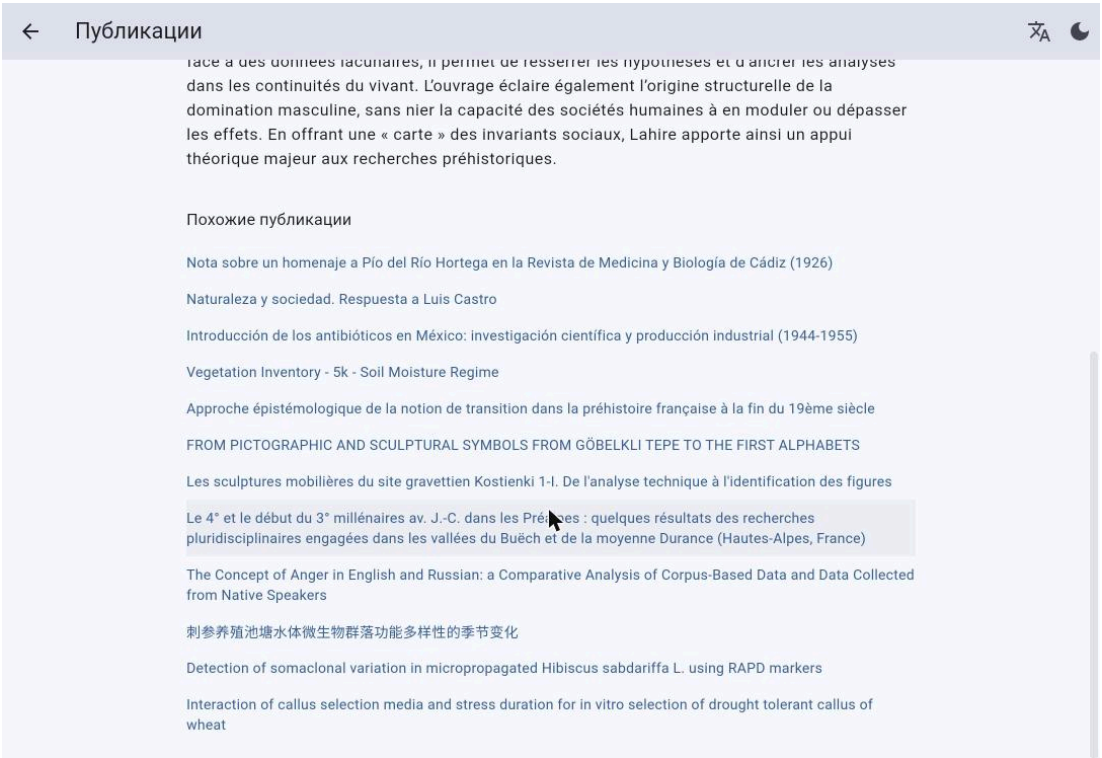


Рисунок 5 – Похожие публикации

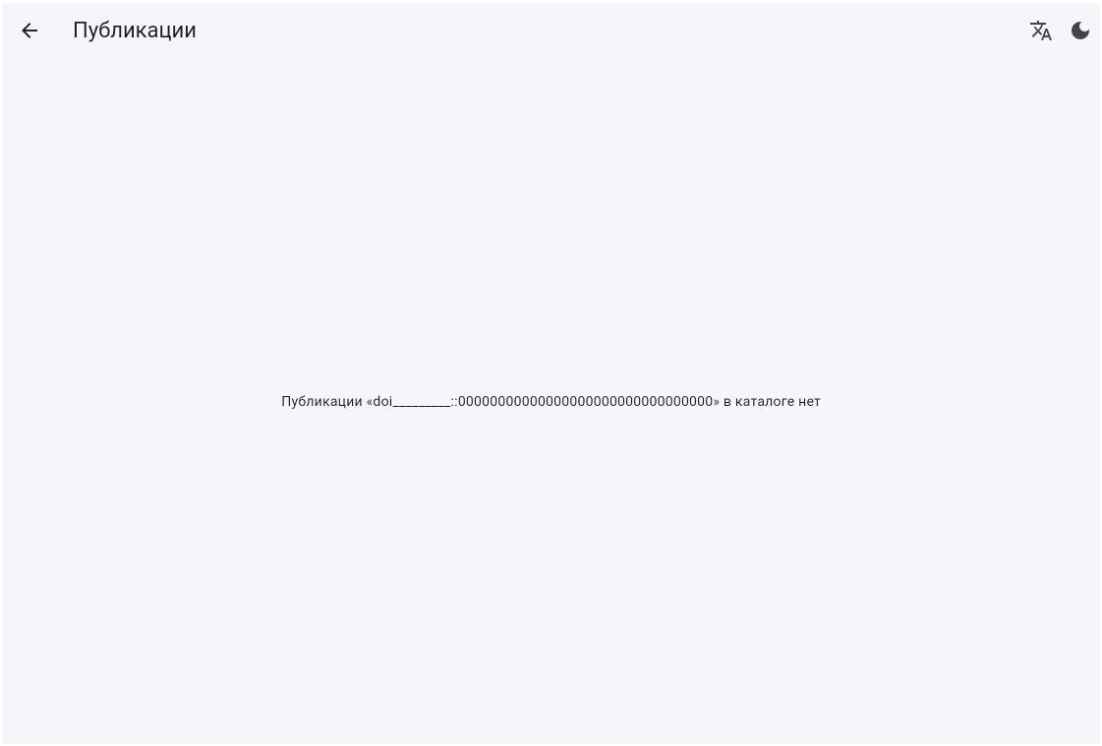


Рисунок 6 – Публикации нет в каталоге

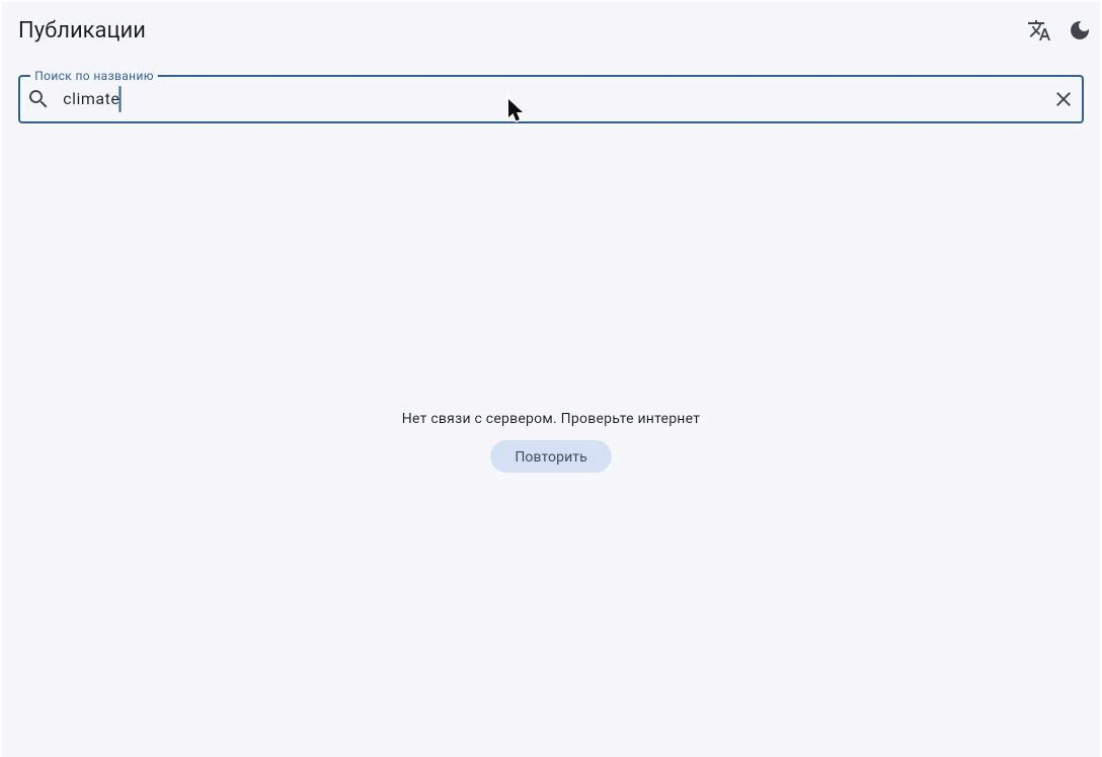


Рисунок 7 – Нет связи с сервером

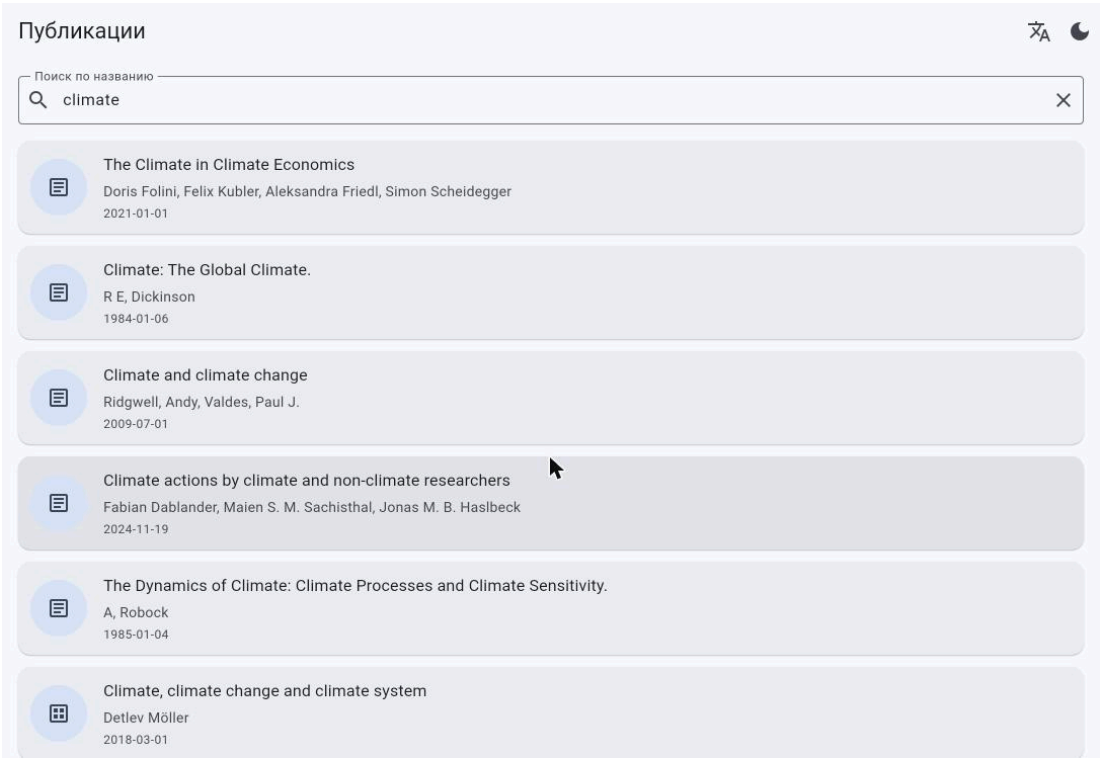


Рисунок 8 – Результаты поиска после «Повторить»

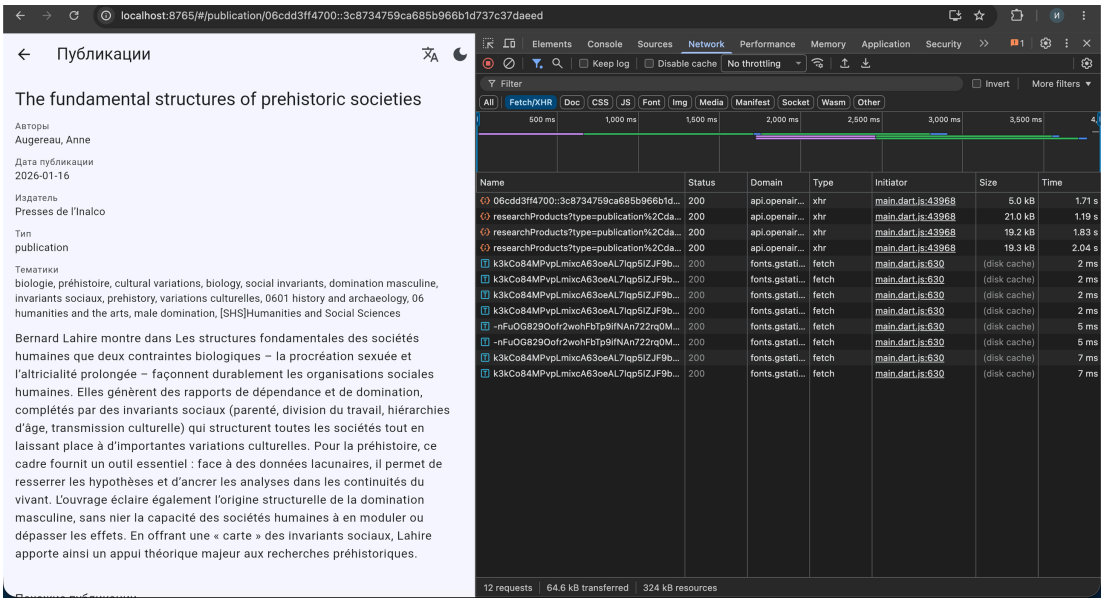


Рисунок 9 – Запросы к серверу во вкладке Network

ЗАКЛЮЧЕНИЕ

В работе каталог публикаций на Flutter переведён с моков на OpenAIRE Graph API. Данные проходят цепочку API-клиент → датасорс → репозиторий → Cubit или Bloc → экран, ответы сервера описаны через DTO на json_serializable, ошибки dio переводятся в sealed-класс LoadException. Список показывает загрузку и ошибки с кнопкой «Повторить», ищет на сервере с паузой 400 мс и подгружает страницы при прокрутке. Карточка грузится своим запросом, похожие публикации загружаются параллельно.

Экраны и состояния второй работы сохранили своё устройство: изменилась папка data, а в состояния добавились варианты ошибки. Приложение работает в браузере и на macOS.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 Flutter : документация / Google. – URL: https://docs.flutter.dev/ (дата обращения: 07.11.2026).

2 dio : документация пакета / pub.dev. – URL: https://pub.dev/packages/dio (дата обращения: 07.11.2026).

3 json_serializable : документация пакета / pub.dev. – URL: https://pub.dev/packages/json_serializable (дата обращения: 07.11.2026).

4 OpenAIRE Graph API : документация / OpenAIRE. – URL: https://graph.openaire.eu/docs/apis/graph-api/ (дата обращения: 07.11.2026).

5 Flutter Bloc Concepts : документация / Bloc Library. – URL: https://bloclibrary.dev/flutter-bloc-concepts/ (дата обращения: 07.11.2026).